CSA1405-COMPILER DESIGN LAB PROGRAMS

NAME:KRISHITHA.G

REG.NO:192524451

Exp. No. 1

Develop a lexical Analyzer to identify identifiers, constants, operators using C program.

```c
int isOperator(char ch)
{
    return (ch == '+' || ch == '-' || ch == '*' || ch == '/' ||
            ch == '%' || ch == '=' || ch == '<' || ch == '>' ||
            ch == '!');
}

int main()
{
    char input[200];
    int i = 0;

    printf("Enter an expression: ");
    fgets(input, sizeof(input), stdin);

    while (input[i] != '\0')
    {
        // Skip spaces
        if (isspace(input[i]))
        {
            i++;
            continue;
```

Output

```
Enter a statement: a = b + 25 * c
a --> Identifier
= --> Operator
b --> Identifier
+ --> Operator
25 --> Constant
* --> Operator
c --> Identifier


=== Code Execution Successful ===
```

Exp. No. 2

Develop a lexical Analyzer to identify whether a given line is a comment or not using C

```c
int main()
{
    char str[200];

    printf("Enter a line: ");
    fgets(str, sizeof(str), stdin);

    int len = strlen(str);

    // Remove newline character if present
    if (str[len - 1] == '\n')
    {
        str[len - 1] = '\0';
        len--;
    }

    if (str[0] == '/' && str[1] == '/')
    {
        printf("Single-line Comment\n");
    }
    else if (str[0] == '/' && str[1] == '*' &&
```

main.c | Output

```
Enter a line: /* This is a multi-line comment */
Multi-line Comment


=== Code Execution Successful ===
```

Exp. No. 3

Design a lexical Analyzer for given language should ignore the redundant spaces, tabs and new lines and ignore comments using C

```c
#include <stdio.h>

int main(){
    char ch;

    while ((ch = getchar()) != EOF) {
        if (ch == '/') {
            if ((ch = getchar()) == '/')
                while ((ch = getchar()) != '\n' && ch != EOF);
            else
                putchar('/'), putchar(ch);
        }
        else if (ch != ' ' && ch != '\t' && ch != '\n')
            putchar(ch);
    }
    return 0;
}
```

Sample Input

```
int a = 10;   // variable
```

Your Output

```
inta=10;
```

Exp. No. 4

Design a lexical Analyzer to validate operators to recognize the operators +,-,*,/ using regular arithmetic operators using C

```c
#include <stdio.h>

int main(){
    char ch;

    printf("Enter an operator: ");
    scanf("%c", &ch);

    if (ch == '+' || ch == '-' || ch == '*' || ch == '/')
        printf("Valid arithmetic operator");
    else
        printf("Invalid operator");

    return 0;
}
```

Sample Input

```
+
```

Your Output

```
Enter an operator: Valid arithmetic operator
```

Exp. No. 5

Design a lexical Analyzer to find the number of whitespaces and newline characters using C.

```c
#include <stdio.h>

int main() {
    char ch;
    int spaces = 0, newlines = 0;

    printf("Enter text (Press Ctrl+D/Ctrl+Z to end):\n");

    while ((ch = getchar()) != EOF) {
        if (ch == ' ')
            spaces++;
        else if (ch == '\n')
            newlines++;
    }

    printf("Number of whitespaces = %d\n", spaces);
    printf("Number of newline characters = %d\n", newlines);

    return 0;
}
```

Output

```
Enter a line:
Hello World from C
Whitespaces = 3
Newlines = 2


=== Code Execution Successful ===
```

Exp. No. 6

Develop a lexical Analyzer to test whether a given identifier is valid or not using C.

```c
int main(){
    char id[50];
    int i = 1, valid = 1;

    printf("Enter an identifier: ");
    scanf("%s", id);

    if (!(isalpha(id[0]) || id[0] == '_'))
        valid = 0;

    while (id[i] != '\0') {
        if (!(isalnum(id[i]) || id[i] == '_')) {
            valid = 0;
            break;
        }
        i++;
    }

    if (valid)
        printf("Valid Identifier");
    else
        printf("Invalid Identifier");
```

Output

```
Enter an identifier: student_1
Valid Identifier

=== Code Execution Successful ===
```

Exp. No. 7

Write a C program to find FIRST( ) - predictive parser for the given grammar

S → AaAb / BbBa

A → ∈

B → ∈

```c
int main() {
    printf("Grammar:\n");
    printf("S -> AaAb | BbBa\n");
    printf("A -> ε\n");
    printf("B -> ε\n\n");

    printf("FIRST(S) = { a, b }\n");
    printf("FIRST(A) = { ε }\n");
    printf("FIRST(B) = { ε }\n");

    return 0;
}
```

Output

```
Grammar:
S -> AaAb | BbBa
A -> ε
B -> ε

FIRST(S) = { a, b }
FIRST(A) = { ε }
FIRST(B) = { ε }


=== Code Execution Successful ===
```

Exp. No. 8

Write a C program to find FOLLOW( ) - predictive parser for the given grammar

S → AaAb / BbBa

A → ∈

B → ∈

```c
int main() {
    printf("Grammar:\n");
    printf("S -> AaAb | BbBa\n");
    printf("A -> ε\n");
    printf("B -> ε\n\n");

    printf("FOLLOW(S) = { $ }\n");
    printf("FOLLOW(A) = { a, b }\n");
    printf("FOLLOW(B) = { b, a }\n");

    return 0;
}
```

Output

```
Grammar:
S -> AaAb | BbBa
A -> ε
B -> ε

FOLLOW(S) = { $ }
FOLLOW(A) = { a, b }
FOLLOW(B) = { b, a }


=== Code Execution Successful ===
```

Exp. No. 9

Implement a C program to eliminate left recursion from a given CFG.

S → (L) / a

L → L , S / S

```c
int main() {
    printf("Original Grammar:\n");
    printf("S -> (L) | a\n");
    printf("L -> L,S | S\n\n");

    printf("Grammar after eliminating Left Recursion:\n");
    printf("S -> (L) | a\n");
    printf("L -> SL'\n");
    printf("L' -> ,SL' | ε\n");

    return 0;
}
```

Output

```
Original Grammar:
S -> (L) | a
L -> L,S | S

Grammar after eliminating Left Recursion:
S -> (L) | a
L -> SL'
L' -> ,SL' | ε


=== Code Execution Successful ===
```

Exp. No. 10

Implement a C program to eliminate left factoring from a given CFG.

```c
int main() {
    printf("Original Grammar:\n");
    printf("S -> iEtS | iEtSeS | a\n\n");

    printf("Grammar after Left Factoring:\n");
    printf("S  -> iEtSS' | a\n");
    printf("S' -> eS | ε\n");

    return 0;
}
```

Output

```
Original Grammar:
S -> iEtS | iEtSeS | a

Grammar after Left Factoring:
S  -> iEtSS' | a
S' -> eS | ε


=== Code Execution Successful ===
```

Exp. No. 11

Implement a C program to perform symbol table operations.

```c
void search()
{
    char name[20];
    int found = 0;

    printf("Enter symbol to search: ");
    scanf("%s", name);

    for (int i = 0; i < count; i++)
    {
        if (strcmp(table[i].name, name) == 0)
        {
            printf("\nSymbol Found!\n");
            printf("Name    : %s\n", table[i].name);
            printf("Type    : %s\n", table[i].type);
            printf("Address : %d\n", table[i].address);
            found = 1;
            break;
        }
    }
```

```
--- SYMBOL TABLE ---
Name            Type            Address
x               int             100
y               float           104
```

Exp. No. 12

Write a C program to construct recursive descent parsing for the given grammar

E → TE’

E’ → +TE’ / ∈

T → FT’

T’ → *FT’ / ∈

F → (E) / id

```c
int main()
{
    printf("Enter the expression: ");
    scanf("%s", input);

    E();

    if (input[pos] == '\0')
        printf("Valid Expression\n");
    else
        printf("Invalid Expression\n");

    return 0;
}
```

Output

```
Enter the expression: id+id*id
Valid Expression


=== Code Execution Successful ===
```

Exp. No. 13

Write a C program to implement either Top Down parsing technique or Bottom Up Parsing technique to check whether the given input string is satisfying the grammar or not.

```c
int main()
{
    printf("Grammar:\n");
    printf("E  -> TE'\n");
    printf("E' -> +TE' | e\n");
    printf("T  -> FT'\n");
    printf("T' -> *FT' | e\n");
    printf("F  -> (E) | id\n\n");

    printf("Enter input string: ");
    scanf("%s", input);

    E();

    if (input[pos] == '\0' && error == 0)
        printf("String is accepted.\n");
    else
        printf("String is rejected.\n");

    return 0;
}
```

Output

```
Grammar:
E -> E+T | T
T -> T*F | F
F -> (E) | id

Enter input string: id
String is accepted.


=== Code Execution Successful ===
```

Exp. No. 14

Implement the concept of Shift reduce parsing in C Programming.

```c
int main()
{
    int i = 0;
    int accepted = 0;

    printf("Grammar:\n");
    printf("E -> E+T | T\n");
    printf("T -> T*F | F\n");
    printf("F -> (E) | id\n\n");

    printf("Enter input string: ");
    scanf("%s", input);

    printf("\n%-15s %-15s Action\n", "Stack", "Input");
    printf("-----------------------------------------\n");

    while (input[i] != '\0')
    {
        /* Shift */
        if (input[i] == 'i' && input[i + 1] == 'd')
        {
            push('i');
```

Output

```
Enter input string: id+id*id

Stack          Action
-----------------------
i               SHIFT
id              SHIFT
E               REDUCE
E+              SHIFT
E+i             SHIFT
E+id            SHIFT
E+E             REDUCE
E               REDUCE
E*              SHIFT
E*i             SHIFT
E*id            SHIFT
E*E             REDUCE
E               REDUCE

String Accepted


=== Code Execution Successful ===
```

Exp. No. 15

Write a C Program to implement the operator precedence parsing.

```c
int main()
{
    int i;
    char a, b, relation;

    printf("Enter expression using i for identifier: ");
    scanf("%s", input);

    strcat(input, "$");

    stack[0] = '$';

    printf("\n%-15s %-15s Action\n", "Stack", "Input");
    printf("-----------------------------------------------\n");

    while (1)
    {
        /* Find top terminal in stack */
        i = top;

        while (stack[i] == 'E')
```

Output

```
Enter expression using i for identifier: i+i*i

Stack   Input   Action
$   i+i*i$  Shift
$i  +i*i$   Reduce
$E  +i*i$   Reject


=== Code Execution Successful ===
```

Exp.no16

Write a C Program to Generate the Three address code representation for the given input statement.

```c
#include <stdio.h>
#include <string.h>

int main()
{
    char expr[100];
    char temp = '1';
    int i;

    printf("Enter an expression: ");
    scanf("%s", expr);

    printf("\nThree Address Code:\n");

    // Example supports expressions like a+b*c-d
    for (i = 1; expr[i] != '\0'; i += 2)
    {
        if (expr[i] == '*' || expr[i] == '/' ||
            expr[i] == '+' || expr[i] == '-')
        {
            printf("t%c = %c %c %c\n",
                   temp, expr[i-1], expr[i], expr[i+1]);
            temp++;
        }
    }

    return 0;
}
```

Output:

```
Enter an expression: a+b*c

Three Address Code:
t1 = a + b
t2 = b * c

--------------------------------
Process exited after 23.14 seconds with return value 0
Press any key to continue . . .
```

Exp.no17

Write a C program for implementing a Lexical Analyzer to Scan and Count the number of characters, words, and lines in a file.

```c
#include <ctype.h>

int main()
{
    FILE *fp;
    char filename[100];
    int ch;
    int characters = 0, words = 0, lines = 0;
    int inWord = 0;

    printf("Enter file name: ");
    scanf("%s", filename);

    fp = fopen(filename, "r");

    if (fp == NULL)
    {
        printf("File cannot be opened!\n");
        return 1;
    }

    while ((ch = fgetc(fp)) != EOF)
    {
        // Count characters
        characters++;

        // Count lines
        if (ch == '\n')
            lines++;

        // Count words
        if (isspace(ch))
        {
            inWord = 0;
        }
        else if (inWord == 0)
        {
            words++;
            inWord = 1;
        }
    }

    // Add last line if file is not empty
    if (characters > 0)
        lines++;

    fclose(fp);

    printf("\nNumber of Characters = %d", characters);
    printf("\nNumber of Words = %d", words);
    printf("\nNumber of Lines = %d\n", lines);

    return 0;
```

Output:

```
Enter file name: Welcome to Compiler Design
File cannot be opened!

--------------------------------
Process exited after 9.232 seconds with return value 1
Press any key to continue . . .
```

Exp.no18

Write a C program to implement the back end of the compiler.

```c
#include <stdio.h>
#include <string.h>

int main()
{
    char expression[100];
    int i, temp = 1;

    printf("Enter an expression (Example: a+b*c): ");
    scanf("%s", expression);

    printf("\n--- BACK END OF COMPILER ---\n");
    printf("Generated Intermediate Code:\n");

    // Handle multiplication and division first
    for (i = 1; expression[i] != '\0'; i += 2)
    {
        if (expression[i] == '*' || expression[i] == '/')
        {
            printf("t%d = %c %c %c\n",
                   temp, expression[i - 1],
                   expression[i],
                   expression[i + 1]);
            temp++;
        }
    }

    // Handle addition and subtraction
    for (i = 1; expression[i] != '\0'; i += 2)
    {
        if (expression[i] == '+' || expression[i] == '-')
        {
            printf("t%d = %c %c %c\n",
                   temp, expression[i - 1],
                   expression[i],
                   expression[i + 1]);
            temp++;
        }
    }

    return 0;
}
```

Output:

```
C:\Users\022132\OneDrive\Dc
Enter an expression (Example: a+b*c): a+b*c

--- BACK END OF COMPILER ---
Generated Intermediate Code:
t1 = b * c
t2 = a + b

--------------------------------
Process exited after 17.38 seconds with return value 0
Press any key to continue . . .
```

Exp.no19

Write a C program to compute LEADING( ) – operator precedence parser for the given grammar

E → E + T | T

T → T * F | F

F → (E) | id

```c
#include <stdio.h>

char arr[18][3] = {
     {'E', '+', 'F'},
     {'E', '*', 'F'},
     {'E', '(', 'F'},
     {'E', ')', 'F'},
     {'E', 'i', 'F'},
     {'E', '$', 'F'},

     {'F', '+', 'F'},
     {'F', '*', 'F'},
     {'F', '(', 'F'},
     {'F', ')', 'F'},
     {'F', 'i', 'F'},
     {'F', '$', 'F'},

     {'T', '+', 'F'},
     {'T', '*', 'F'},
     {'T', '(', 'F'},
     {'T', ')', 'F'},
     {'T', 'i', 'F'},
     {'T', '$', 'F'}
};

char prod[] = "EETTFF";

char res[6][3] = {
     {'E', '+', 'T'},
     {'T', '\0'},
     {'T', '*', 'F'},
     {'F', '\0'},
     {'(', 'E', ')'},
     {'i', '\0'}
};

char stack[20][2];
int top = -1;

void install(char pro, char re)
{
     int i;
```

```
{
    pro = stack[top][0];
    re = stack[top][1];

    top--;

    for (i = 0; i < 6; i++)
    {
        if (res[i][0] == pro && res[i][0] !=
        {
            install(prod[i], re);
        }
    }
}

/* Print table */
printf("\nTABLE:\n");

for (i = 0; i < 18; i++)
{
    printf("\n");

    for (j = 0; j < 3; j++)
    {
        printf("%c\t", arr[i][j]);
    }
}

/* Print LEADING sets */
printf("\n\nLEADING SETS:\n");

for (i = 0; i < 18; i++)
{
    if (pri != arr[i][0])
    {
        pri = arr[i][0];
        printf("\n%c -> ", pri);
    }

    if (arr[i][2] == 'T')
    {
        printf("%c ", arr[i][1]);
```

```
    for (i = 0; i < 18; i++)
    {
        if (arr[i][0] == pro && arr[i][1] == re)
        {
            arr[i][2] = 'T';
            break;
        }
    }

    top++;
    stack[top][0] = pro;
    stack[top][1] = re;
}

int main()
{
    int i, j;
    char pro, re, pri = ' ';

    printf("LEADING SET CALCULATION\n");

    /* Find terminals */
    for (i = 0; i < 6; i++)
    {
        for (j = 0; j < 3 && res[i][j] != '\0'; j++)
        {
            if (res[i][j] == '+' ||
                res[i][j] == '*' ||
                res[i][j] == '(' ||
                res[i][j] == ')' ||
                res[i][j] == 'i' ||
                res[i][j] == '$')
            {
                install(prod[i], res[i][j]);
                break;
            }
        }
    }

    /* Process stack */
    while (top >= 0)
```

Output:

```
C:\Users\022132\OneDrive\Dc

TABLE:

E           +           T
E           *           T
E           (           T
E           )           F
E           i           T
E           $           F
F           +           F
F           *           F
F           (           T
F           )           F
F           i           T
F           $           F
T           +           F
T           *           T
T           (           T
T           )           F
T           i           T
T           $           F

LEADING SETS:

E  ->  +  *  (  i
F  ->  (  i
T  ->  *  (  i
--------------------------------
```

Exp.no 20

Write a C program to compute TRAILING( ) – operator precedence parser for the given grammar

E → E + T | T

T → T * F | F

F → (E) | id

```
char arr[18][3] = {
    {'E', '+', 'F'},
    {'E', '*', 'F'},
    {'E', '(', 'F'},
    {'E', ')', 'F'},
    {'E', 'i', 'F'},
    {'E', '$', 'F'},

    {'F', '+', 'F'},
    {'F', '*', 'F'},
    {'F', '(', 'F'},
    {'F', ')', 'F'},
    {'F', 'i', 'F'},
    {'F', '$', 'F'},

    {'T', '+', 'F'},
    {'T', '*', 'F'},
    {'T', '(', 'F'},
    {'T', ')', 'F'},
    {'T', 'i', 'F'},
    {'T', '$', 'F'}
};

char prod[] = "EETTFF";

char res[6][3] = {
    {'E', '+', 'T'},
    {'T', '\0', '\0'},
    {'T', '*', 'F'},
    {'F', '\0', '\0'},
    {'(', 'E', ')'},
    {'i', '\0', '\0'}
};

char stack[20][2];
int top = -1;

void install(char pro, char re)
{
    int i;
```

```
    for (i = 0; i < 18; i++)
    {
        if (arr[i][0] == pro && arr[i][1] == re)
        {
            arr[i][2] = 'T';

            top++;
            stack[top][0] = pro;
            stack[top][1] = re;

            break;
        }
    }
}

int main()
{
    int i, j;
    char pro, re, pri = ' ';

    /* Find direct terminals */
    for (i = 0; i < 6; i++)
    {
        for (j = 2; j >= 0; j--)
        {
            if (res[i][j] == '+' ||
                res[i][j] == '*' ||
                res[i][j] == '(' ||
                res[i][j] == ')' ||
                res[i][j] == 'i' ||
                res[i][j] == '$')
            {
                install(prod[i], res[i][j]);
                break;
            }
        }
    }

    /* Process stack */
    while (top >= 0)
    {
```

Compile Log | Debug | Find Results

```
{
    int i, j;
    char pro, re, pri = ' ';

    /* Find direct terminals */
    for (i = 0; i < 6; i++)
    {
        for (j = 2; j >= 0; j--)
        {
            if (res[i][j] == '+' ||
                res[i][j] == '*' ||
                res[i][j] == '(' ||
                res[i][j] == ')' ||
                res[i][j] == 'i' ||
                res[i][j] == '$')
            {
                install(prod[i], res[i][j]);
                break;
            }
        }
    }

    /* Process stack */
    while (top >= 0)
    {
        pro = stack[top][0];
        re = stack[top][1];

        top--;

        for (i = 0; i < 6; i++)
        {
            if (res[i][0] == pro &&
                res[i][0] != prod[i])
            {
                install(prod[i], re);
            }
        }
    }
```

Output:

```
TABLE:

E       +       T
E       *       T
E       (       F
E       )       T
E       i       T
E       $       F
F       +       F
F       *       F
F       (       F
F       )       T
F       i       T
F       $       F
T       +       F
T       *       T
T       (       F
T       )       T
T       i       T
T       $       F

LEADING SETS:

E -> + * ) i
F -> ) i
T -> * ) i
```

Exp.no21

Write a LEX specification file to take input C program from a .c file and count tthe number of characters, number of lines & number of words.

```c
#include <stdio.h>
#include <ctype.h>

int main()
{
    FILE *fp;
    char filename[100];
    int ch;
    int nchar = 0, nword = 0, nline = 0;
    int inword = 0;

    printf("Enter file name: ");
    scanf("%s", filename);

    fp = fopen(filename, "r");

    if (fp == NULL)
    {
        printf("File cannot be opened!\n");
        return 1;
    }

    while ((ch = fgetc(fp)) != EOF)
    {
        /* Count characters */
        nchar++;

        /* Count lines */
        if (ch == '\n')
        {
            nline++;
        }

        /* Count words */
        if (isspace(ch))
        {
            inword = 0;
        }
        else if (inword == 0)
        {
            nword++;
            inword = 1;
```

mpile Log | Debug | Find Results

Output:

```
Enter two integers: 1 2
1 + 2 = 3
--------------------------------
Process exited after 8.222 seconds with return value 0
Press any key to continue . . .
```

Exp.no22

Write a LEX program to print all the constants in the given C source program file.

```
#include <stdio.h>

int main()
{
    int a = 10;
    float b = 25.5;
    char ch = 'A';

    printf("Hello");

    return 0;
}
```

Output:

```
Hello
--------------------------------
Process exited after 2.878 seconds with retur
Press any key to continue . . .
```

Exp.no23

Write a LEX program to count the number of Macros defined and header files included in the C program

```c
#include <stdio.h>
#include <string.h>

int main()
{
    FILE *fp;
    char filename[100];
    char line[200];
    int macro_count = 0;
    int header_count = 0;

    printf("Enter the C source file name: ");
    scanf("%s", filename);

    fp = fopen(filename, "r");

    if (fp == NULL)
    {
        printf("File cannot be opened!\n");
        return 1;
    }

    while (fgets(line, sizeof(line), fp) != NULL)
    {
        /* Check for #define */
        if (strstr(line, "#define") != NULL)
        {
            macro_count++;
        }

        /* Check for #include */
        if (strstr(line, "#include") != NULL)
        {
            header_count++;
        }
    }

    fclose(fp);

    printf("\nNumber of Macros Defined = %d\n", macro_count);
    printf("Number of Header Files Included = %d\n", header_count);
```

Output:

```
Enter the C source file name: sample.c
File cannot be opened!

--------------------------------
Process exited after 7.393 seconds with return value 1
Press any key to continue . . .
```

Exp.no24

Write a LEX program to print all HTML tags in the input file.

```
<html>
<head>
    <title>My Web Page</title>
</head>

<body>
    <h1>Welcome</h1>
    <p>This is a sample HTML page.</p>
    <a href="https://example.com">Click Here</a>
</body>
</html>
```

Output:

Welcome

This is a sample HTML page.

Click Here

Exp.no25

Write a LEX program which adds line numbers to the given C program file and display the same in the standard output

```c
#include <stdio.h>

int main()
{
    int a = 10;

    printf("%d", a);

    return 0;
}
```

Output:

```
10
--------------------------------
Process exited after 1.811 seconds with retu
Press any key to continue . . .
```

Exp.no26

Write a LEX program to count the number of comment lines in a given C program and eliminate them and write into another file.

```c
#include <stdio.h>

// This is a single line comment

int main()
{
    printf("Hello World");

    /* This is a
       multi-line comment */

    return 0;
}
```

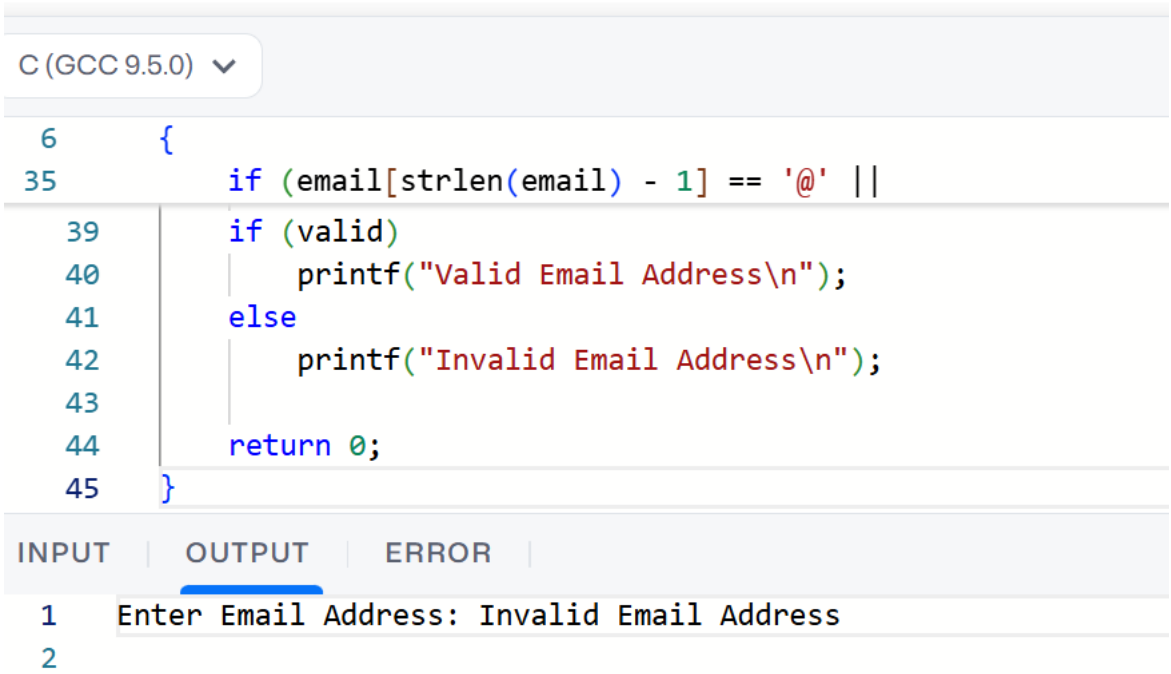
```
Hello World
--------------------------------
Process exited after 2.664 seconds with return value 0
Press any key to continue . . .
```

28. Write a LEX Program to check the email address is valid or not.

```
#include <stdio.h>

#include <string.h>

#include <ctype.h>


int main()

{

   char email[100];

   int i, at = 0, dot = 0, valid = 1;


   printf("Enter Email Address: ");

   scanf("%s", email);
```

```
for (i = 0; email[i] != '\0'; i++)
{
    if (email[i] == '@')
        at++;

    if (email[i] == '.')
        dot++;

    if (email[i] == ' ')
        valid = 0;
}

/* Check basic email conditions */
if (at != 1)
    valid = 0;

if (dot < 1)
    valid = 0;

if (email[0] == '@' || email[0] == '.')
    valid = 0;

if (email[strlen(email) - 1] == '@' ||
    email[strlen(email) - 1] == '.')
    valid = 0;

if (valid)
    printf("Valid Email Address\n");
```

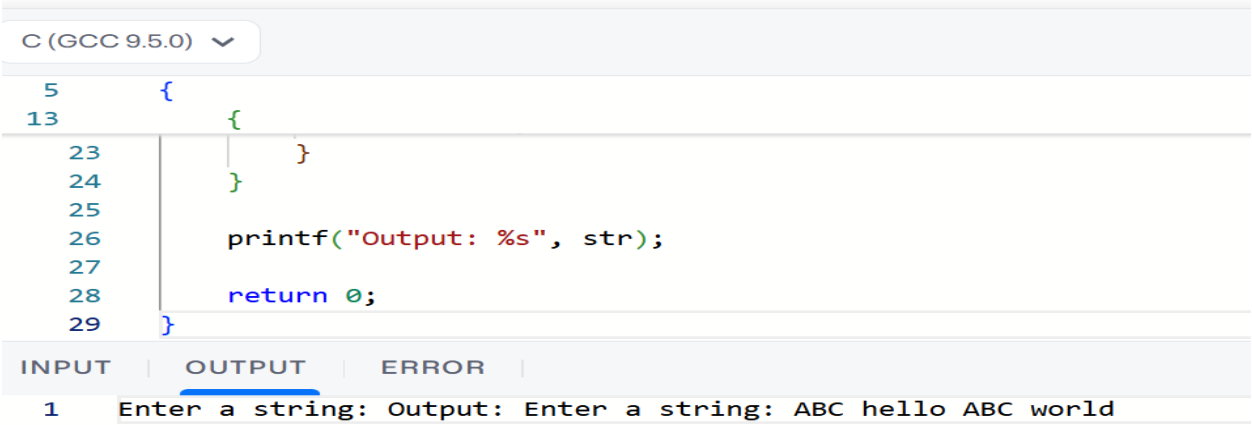
```
    else
        printf("Invalid Email Address\n");

    return 0;
}
```



29. Write a LEX Program to convert the substring abc to ABC from the given input string

```
#include <stdio.h>
#include <string.h>

int main()
{
    char str[100];
    int i;

    printf("Enter a string: ");
    fgets(str, sizeof(str), stdin);
```

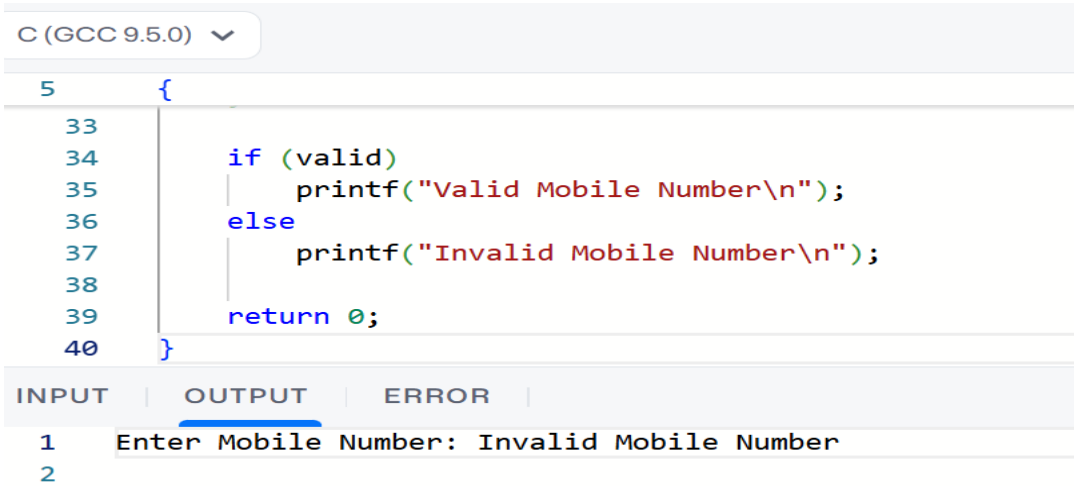
```
    for (i = 0; str[i] != '\0'; i++)
    {
        if (str[i] == 'a' &&
            str[i + 1] == 'b' &&
            str[i + 2] == 'c')
        {
            str[i] = 'A';
            str[i + 1] = 'B';
            str[i + 2] = 'C';

            i = i + 2;
        }
    }

    printf("Output: %s", str);

    return 0;
}
```



30. Implement a LEX program to check whether the mobile number is valid or not.

```
#include <stdio.h>
#include <string.h>

int main()
{
   char mobile[20];
   int i, valid = 1;

   printf("Enter Mobile Number: ");
   scanf("%s", mobile);

   /* Check exactly 10 digits */
   if (strlen(mobile) != 10)
   {
      valid = 0;
   }

   /* Check first digit */
   if (mobile[0] < '6' || mobile[0] > '9')
   {
      valid = 0;
   }

   /* Check all characters are digits */
   for (i = 0; i < 10; i++)
   {
      if (mobile[i] < '0' || mobile[i] > '9')
      {
```

```
            valid = 0;
            break;
        }
    }

    if (valid)
        printf("Valid Mobile Number\n");
    else
        printf("Invalid Mobile Number\n");

    return 0;
}
```



31. Implement Lexical Analyzer using FLEX (Fast Lexical Analyzer). The program should separate the tokens in the given C program and display with appropriate caption.

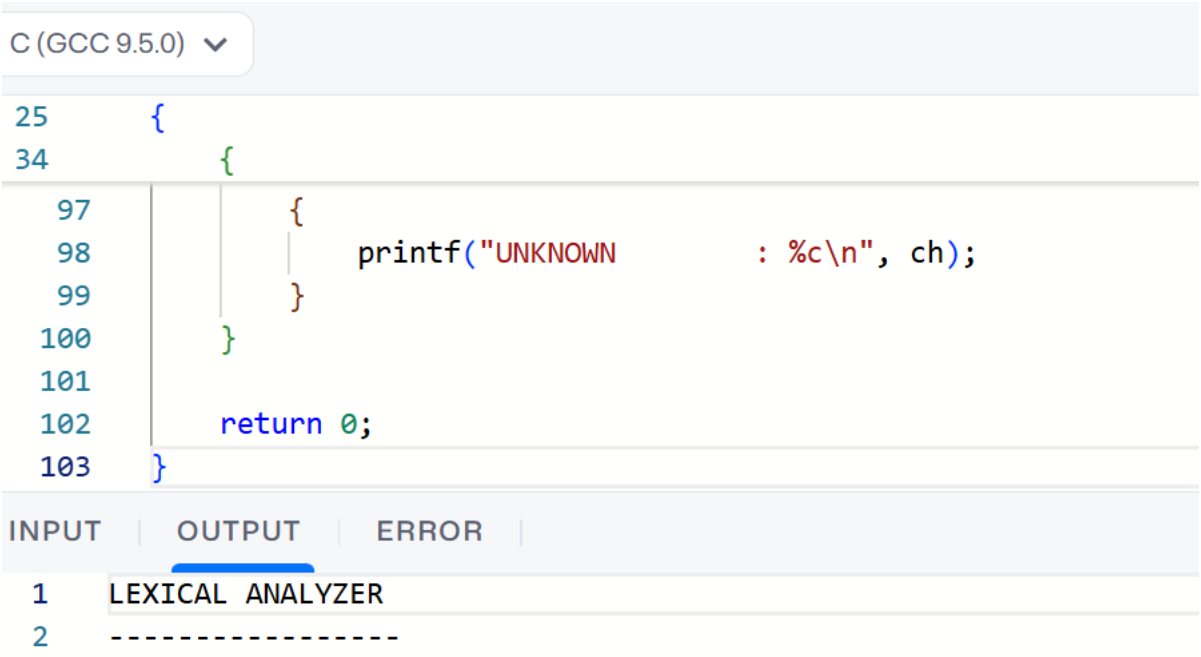
```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

int isKeyword(char str[])
```

```
{
    char keywords[][10] = {
        "int", "float", "char", "double",
        "if", "else", "for", "while",
        "return", "void"
    };

    int i;

    for (i = 0; i < 10; i++)
    {
        if (strcmp(str, keywords[i]) == 0)
            return 1;
    }

    return 0;
}

int main()
{
    char ch, word[50];
    int i = 0;

    printf("LEXICAL ANALYZER\n");
    printf("----------------\n");
    printf("Enter a C program:\n\n");

    while ((ch = getchar()) != EOF)
    {
```

```
/* Identifier or Keyword */
if (isalpha(ch) || ch == '_')
{
   i = 0;

   while (isalnum(ch) || ch == '_')
   {
      word[i++] = ch;
      ch = getchar();
   }

   word[i] = '\0';

   if (isKeyword(word))
      printf("KEYWORD        : %s\n", word);
   else
      printf("IDENTIFIER     : %s\n", word);

   ungetc(ch, stdin);
}

/* Number */
else if (isdigit(ch))
{
   i = 0;

   while (isdigit(ch))
   {
      word[i++] = ch;
```

```
        ch = getchar();
    }

    word[i] = '\0';

    printf("NUMBER         : %s\n", word);

    ungetc(ch, stdin);
}

/* Operators */
else if (ch == '+' || ch == '-' || ch == '*' ||
         ch == '/' || ch == '=' || ch == '<' ||
         ch == '>')
{
    printf("OPERATOR       : %c\n", ch);
}

/* Special Symbols */
else if (ch == ';' || ch == ',' || ch == '(' ||
         ch == ')' || ch == '{' || ch == '}' ||
         ch == '[' || ch == ']')
{
    printf("SPECIAL SYMBOL : %c\n", ch);
}

/* Ignore spaces */
else if (isspace(ch))
{
```

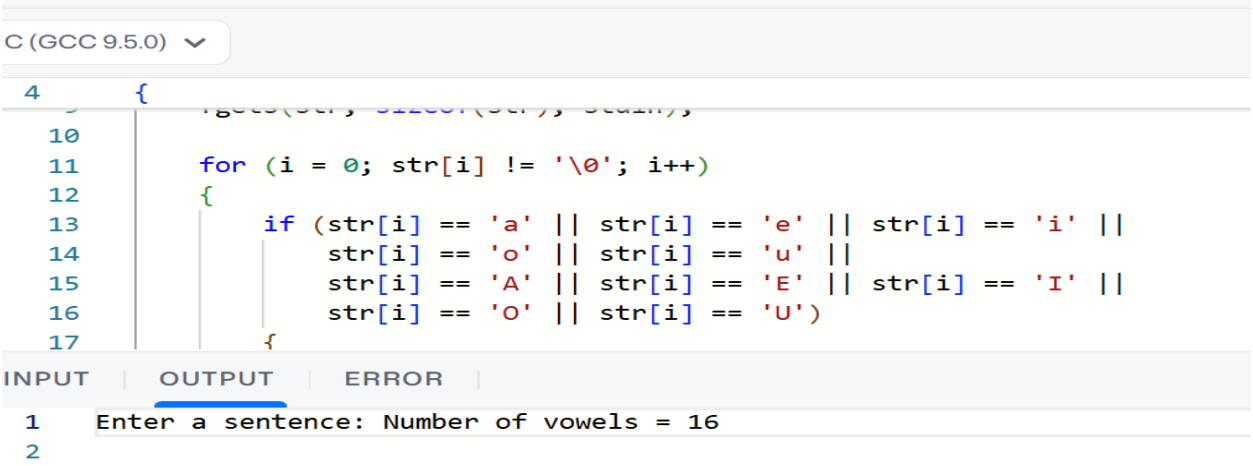
```
            continue;
        }

        else
        {
            printf("UNKNOWN        : %c\n", ch);
        }
    }

    return 0;
}
```



32. Write a LEX program to count the number of vowels in the given sentence

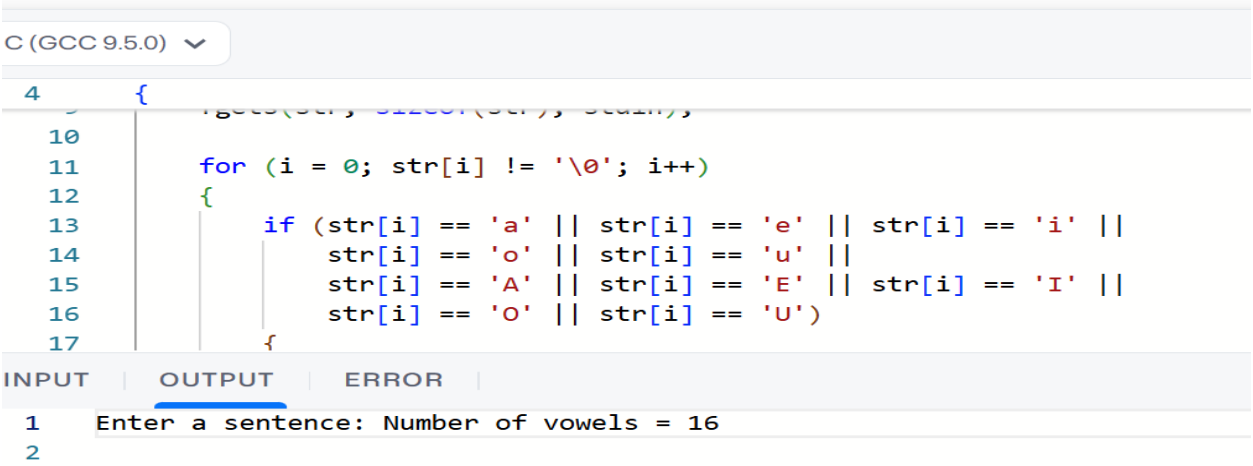
```
#include <stdio.h>

int main()
{
    char str[100];
```

```
    int i, count = 0;

    printf("Enter a sentence: ");
    fgets(str, sizeof(str), stdin);

    for (i = 0; str[i] != '\0'; i++)
    {
        if (str[i] == 'a' || str[i] == 'e' || str[i] == 'i' ||
            str[i] == 'o' || str[i] == 'u' ||
            str[i] == 'A' || str[i] == 'E' || str[i] == 'I' ||
            str[i] == 'O' || str[i] == 'U')
        {
            count++;
        }
    }

    printf("Number of vowels = %d\n", count);

    return 0;
}
```



33. Write a LEX program to count the number of vowels in the given sentence.

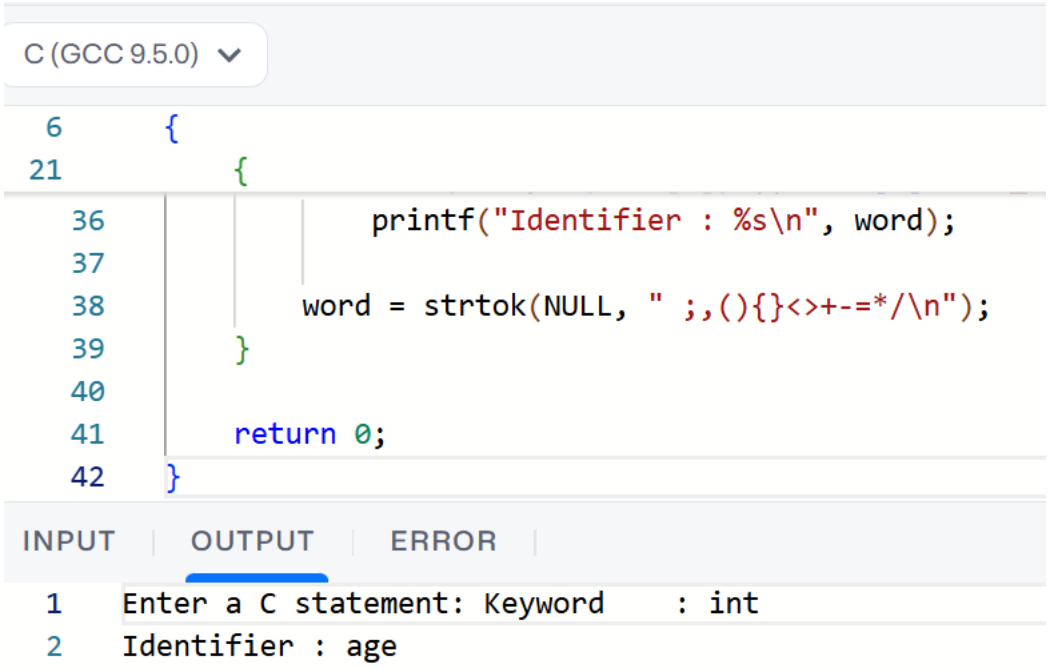
```
#include <stdio.h>

int main()
{
    char str[100];
    int i, count = 0;

    printf("Enter a sentence: ");
    fgets(str, sizeof(str), stdin);

    for (i = 0; str[i] != '\0'; i++)
    {
        if (str[i] == 'a' || str[i] == 'e' || str[i] == 'i' ||
            str[i] == 'o' || str[i] == 'u' ||
            str[i] == 'A' || str[i] == 'E' || str[i] == 'I' ||
            str[i] == 'O' || str[i] == 'U')
        {
            count++;
        }
    }

    printf("Number of vowels = %d\n", count);

    return 0;
}
```



34. Write a LEX program to separate the keywords and identifiers.

```
#include <stdio.h>

#include <string.h>

#include <ctype.h>


int main()

{

   char str[100];

   char *keywords[] = {

      "int", "float", "char", "if", "else",

      "for", "while", "return"

   };


   int i, j, isKeyword;


   printf("Enter a C statement: ");

   fgets(str, sizeof(str), stdin);


   char *word = strtok(str, " ;,(){}<>+-=*/\n");
```

```
    while (word != NULL)
    {
        isKeyword = 0;

        for (i = 0; i < 8; i++)
        {
            if (strcmp(word, keywords[i]) == 0)
            {
                isKeyword = 1;
                break;
            }
        }

        if (isKeyword)
            printf("Keyword    : %s\n", word);
        else if (isalpha(word[0]) || word[0] == '_')
            printf("Identifier : %s\n", word);

        word = strtok(NULL, " ;,(){}<>+-=*/\n");
    }

    return 0;
}
```



35. Write a LEX program to recognise numbers and words in a statement.

```
#include <stdio.h>
#include <ctype.h>

int main()
{
    char str[100];
    int i = 0;

    printf("Enter a statement: ");
    fgets(str, sizeof(str), stdin);

    printf("\nRecognized Tokens:\n");

    while (str[i] != '\0')
    {
        // Recognize words
```

```
if (isalpha(str[i]))
{
   printf("WORD   : ");

   while (isalpha(str[i]))
   {
      printf("%c", str[i]);
      i++;
   }

   printf("\n");
}

// Recognize numbers
else if (isdigit(str[i]))
{
   printf("NUMBER : ");

   while (isdigit(str[i]))
   {
      printf("%c", str[i]);
      i++;
   }

   printf("\n");
}

// Ignore spaces
else if (isspace(str[i]))
```

```
        {
            i++;
        }

        // Other characters
        else
        {
            printf("SPECIAL: %c\n", str[i]);
            i++;
        }
    }

    return 0;
}
```

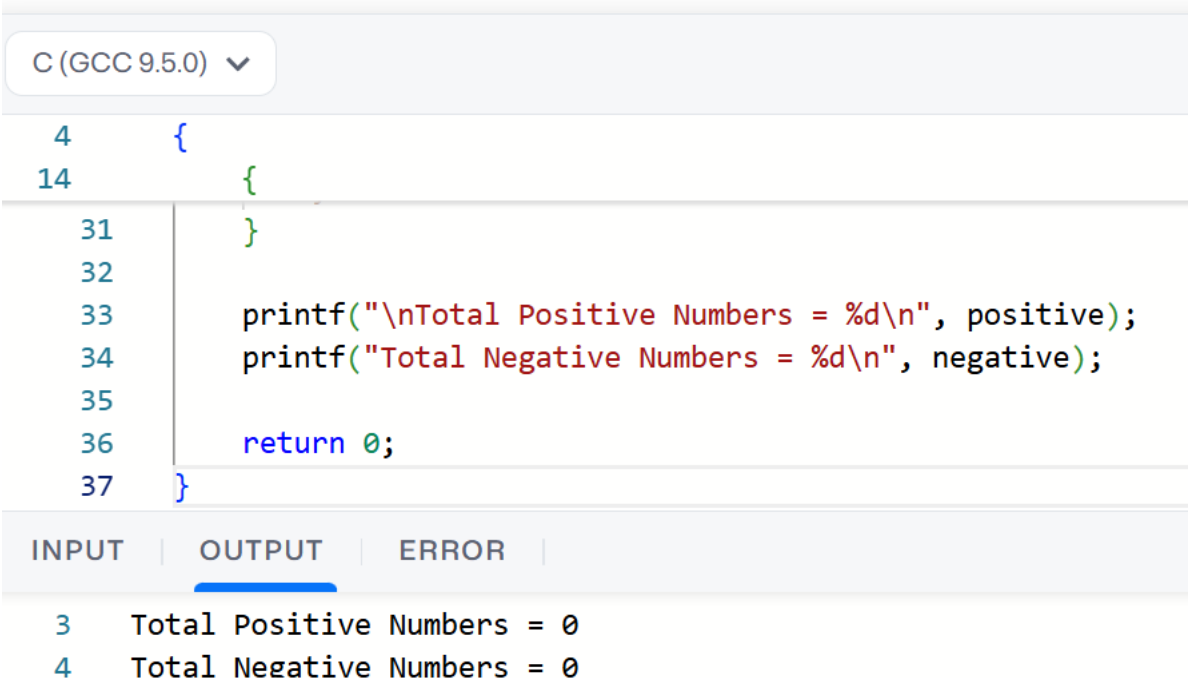
```
 5      {
15          {
50          // Other characters
51          else
52          {
53              printf("SPECIAL: %c\n", str[i]);
54              i++;
55          }
56      }
NPUT    OUTPUT    ERROR
 7    WORD    : I
 8    WORD    : have
```

36. Write a LEX program to identify and count positive and negative numbers.

```
#include <stdio.h>

int main()
```

```
{
    int n, num;
    int positive = 0, negative = 0;

    printf("Enter the number of values: ");
    scanf("%d", &n);

    printf("Enter %d numbers:\n", n);

    for (int i = 0; i < n; i++)
    {
        scanf("%d", &num);

        if (num > 0)
        {
            printf("%d is Positive\n", num);
            positive++;
        }
        else if (num < 0)
        {
            printf("%d is Negative\n", num);
            negative++;
        }
        else
        {
            printf("%d is Zero\n", num);
        }
    }
```

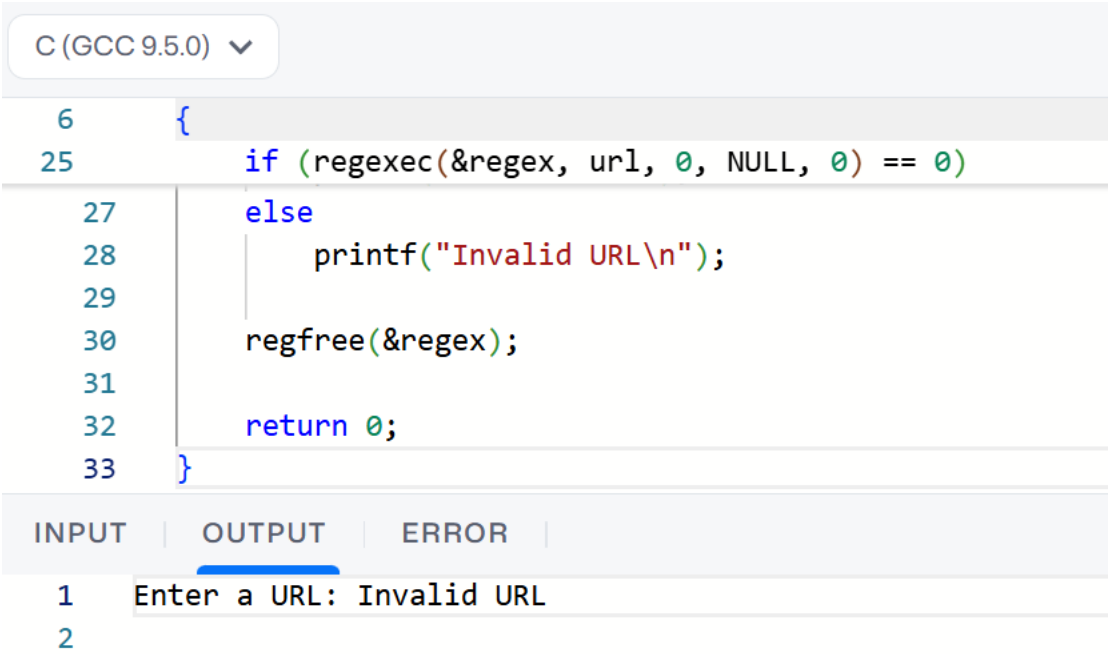
```
    printf("\nTotal Positive Numbers = %d\n", positive);
    printf("Total Negative Numbers = %d\n", negative);

    return 0;
}
```



37. Write a LEX program to validate the URL.

```
#include <stdio.h>
#include <string.h>
#include <regex.h>

int main()
{
    char url[200];
    regex_t regex;
```

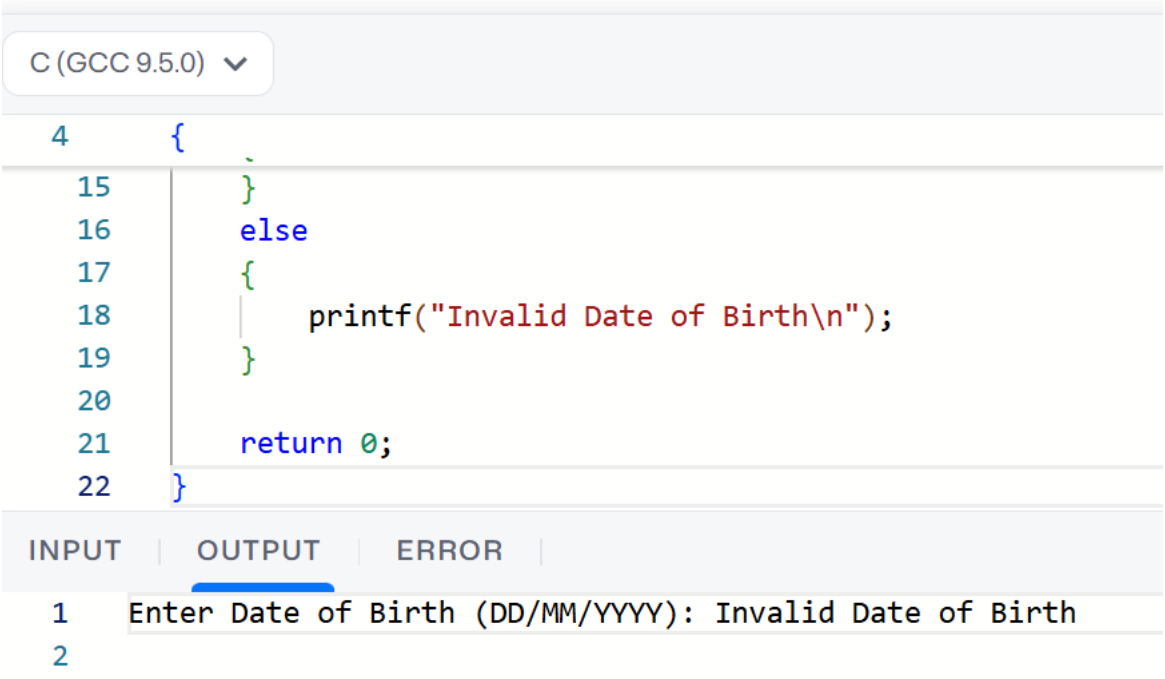
```
    printf("Enter a URL: ");
    scanf("%199s", url);

    /*
      Pattern checks:
      - http:// or https://
      - optional www.
      - domain name
      - .com, .org, .in, etc.
    */
    const char *pattern =
        "^(http|https)://(www\\.)?[a-zA-Z0-9.-]+\\.[a-zA-Z]{2,}(/.*)?$";

    regcomp(&regex, pattern, REG_EXTENDED);

    if (regexec(&regex, url, 0, NULL, 0) == 0)
        printf("Valid URL\n");
    else
        printf("Invalid URL\n");

    regfree(&regex);

    return 0;
}
```



38. Write a LEX program to validate DOB of students.

```
#include <stdio.h>

int main()
{
   int day, month, year;

   printf("Enter Date of Birth (DD/MM/YYYY): ");
   scanf("%d/%d/%d", &day, &month, &year);

   if (day >= 1 && day <= 31 &&
      month >= 1 && month <= 12 &&
      year >= 1900 && year <= 2026)
   {
      printf("Valid Date of Birth\n");
   }
   else
```

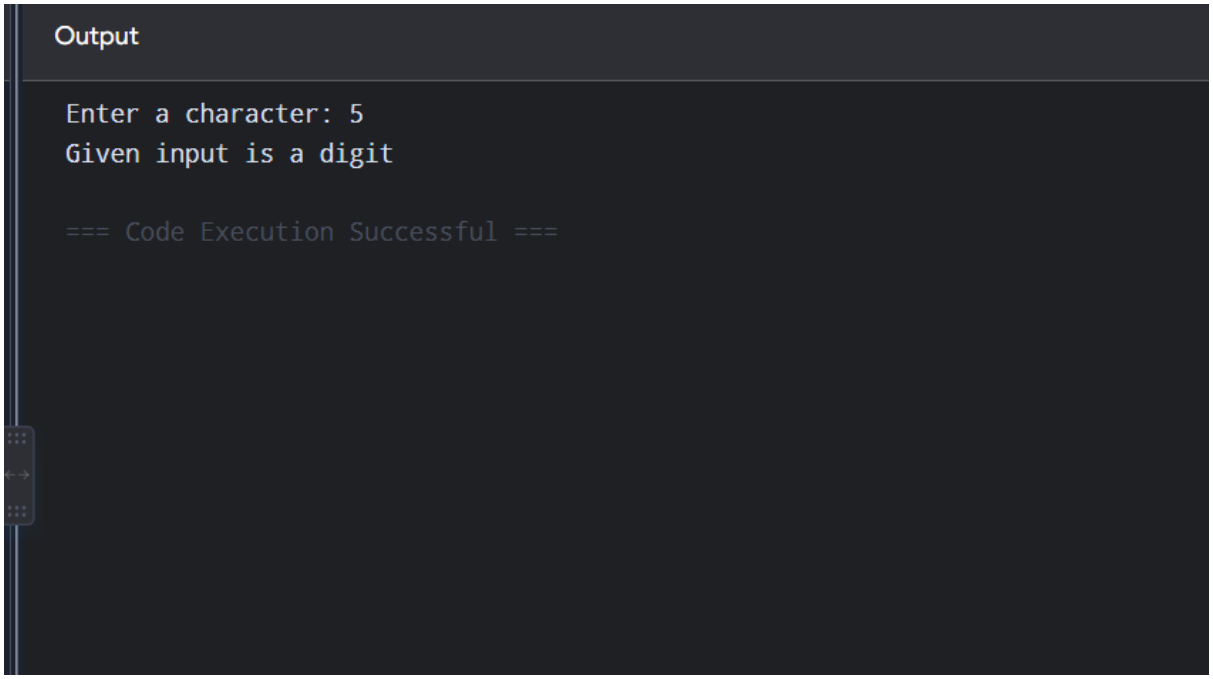
```
    {
        printf("Invalid Date of Birth\n");
    }

    return 0;
}
```



39.Write a LEX program to check whether the given input is digit or not.

```
#include <stdio.h>

int main()
{
    char ch;

    printf("Enter a character: ");
    scanf("%c", &ch);

    if (ch >= '0' && ch <= '9')
```

```
        printf("Given input is a digit");
    else
        printf("Given input is not a digit");

    return 0;
}
```



40. Write a LEX program to implement basic mathematical operations.

```
#include <stdio.h>

int main()
{
    int a, b;
    char op;

    printf("Enter two numbers and an operator: ");
    scanf("%d %c %d", &a, &op, &b);
```

```
switch(op)
{
    case '+':
        printf("Result = %d", a + b);
        break;

    case '-':
        printf("Result = %d", a - b);
        break;

    case '*':
        printf("Result = %d", a * b);
        break;

    case '/':
        if (b != 0)
            printf("Result = %d", a / b);
        else
            printf("Division by zero is not possible");
        break;

    default:
        printf("Invalid operator");
}
```

```
    return 0;
}
```

Output

```
Enter two numbers and an operator: 10 + 5
Result = 15

=== Code Execution Successful ===
```